

ASSIGNMENT 1:

1. API Data Retrieval and Storage: You are tasked with fetching data from an external REST API, storing it in a local SQLite database, and displaying the retrieved data. The API provides a list of books in JSON format with attributes like title, author, and publication year.

```
import sqlite3
import requests
API_URL = "https://openlibrary.org/search.json"
DB_FILE = "books.db"
def fetch_books(query="python", limit=20):
    params = {
        "q": query,
        "limit": limit,
        "fields": "title,author_name,first_publish_year",
    }
    resp = requests.get(API_URL, params=params, timeout=10)
    resp.raise_for_status()
    return resp.json().get("docs", [])

def init_db(conn):
    conn.execute("""
        CREATE TABLE IF NOT EXISTS books (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            author TEXT,
            publication_year INTEGER,
            UNIQUE(title, author)
        )
    """)

def store_books(conn, books):
    rows = [
        (
            b.get("title"),
            ", ".join(b.get("author_name", [])) or None,
        )
```

```

        b.get("first_publish_year"),
    )
    for b in books
        if b.get("title")
    ]
    conn.executemany(
        "INSERT OR IGNORE INTO books (title, author,
publication_year) VALUES (?, ?, ?)",
        rows,
    )
    conn.commit()

def display_books(conn):
    rows = conn.execute(
        "SELECT title, author, publication_year FROM books ORDER BY
publication_year"
    ).fetchall()
    print(f"{'Title':<50} {'Author':<30} {'Year':<6}")
    print("-" * 88)
    for title, author, year in rows:
        print(f"{title[:48]:<50} {(author or 'Unknown')[:28]:<30}
{year or 'N/A':<6}")
    print(f"\nTotal records: {len(rows)}")

def main():
    conn = sqlite3.connect(DB_FILE)
    try:
        init_db(conn)
        store_books(conn, fetch_books())
        display_books(conn)
    except requests.RequestException as e:
        print(f"API request failed: {e}")
    finally:
        conn.close()

if __name__ == "__main__":
    main()

```

2.Data Processing and Visualization: Given a dataset containing information about students' test scores, fetch the data from an API, calculate the average score, and create a bar chart to visualize the data.

```
import requests
import pandas as pd
import matplotlib.pyplot as plt
API_URL = "https://datasets-server.huggingface.co/rows"
PARAMS = {
    "dataset": "mstz/student_performance",
    "config": "math",
    "split": "train",
}
PAGE_SIZE = 100
def fetch_students():
    rows, offset = [], 0
    while True:
        resp = requests.get(
            API_URL, params={**PARAMS, "offset": offset, "length":
PAGE_SIZE}, timeout=15
        )
        resp.raise_for_status()
        data = resp.json()
        rows += [r["row"] for r in data["rows"]]
        offset += PAGE_SIZE
        if offset >= data["num_rows_total"]:
            return pd.DataFrame(rows)
def main():
    df = fetch_students()
    df["avg_score"] = df[["reading_score",
"writing_score"]].mean(axis=1)
    print(f"Students: {len(df)}")
    print(f"Average reading score: {df['reading_score'].mean():.2f}")
    print(f"Average writing score: {df['writing_score'].mean():.2f}")
    overall = df["avg_score"].mean()
    print(f"Overall average score: {overall:.2f}")
```

```

    by_group =
df.groupby("ethnicity")["avg_score"].mean().sort_index()
    ax = by_group.plot(kind="bar", color="steelblue", figsize=(8, 5),
rot=0)
    ax.axhline(overall, color="red", linestyle="--", label=f"Overall
avg ({overall:.1f})")
    ax.bar_label(ax.containers[0], fmt="%.1f")
    ax.set_title("Average Test Score by Student Group")
    ax.set_xlabel("Group")
    ax.set_ylabel("Average score (reading + writing)")
    ax.set_ylim(0, 100)
    ax.legend()
    plt.tight_layout()
    plt.savefig("scores.png", dpi=150)
    plt.show()
if __name__ == "__main__":
    main()

```

3. CSV Data Import to a Database: Write a Python script that reads data from a CSV file containing user information (e.g., name, email) and inserts it into a SQLite database.

```

import csv

import sqlite3

CSV_FILE = "users.csv"

DB_FILE = "users.db"

conn = sqlite3.connect(DB_FILE)

cursor = conn.cursor()

cursor.execute("""

```

```

CREATE TABLE IF NOT EXISTS users (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    name TEXT NOT NULL,

    email TEXT NOT NULL

)

"""

with open(CSV_FILE, "r", newline="", encoding="utf-8") as file:

    reader = csv.DictReader(file)

    for row in reader:

        cursor.execute("""

            INSERT INTO users (name, email)

            VALUES (?, ?)

            """, (row["name"], row["email"]))

conn.commit()

cursor.execute("SELECT id, name, email FROM users")

print("\nUsers stored in database:")

for user_id, name, email in cursor.fetchall():

    print(f"ID: {user_id} | Name: {name} | Email: {email}")

conn.close()

```

4. Send a link to the most complex Python code you have written

<https://github.com/KhushieSharma/Factfinder>

5. Send a link to the most complex database code you have written

<https://github.com/KhushieSharma/studybuddy>

ASSIGNMENT-2:

1. Where would I rate myself?

I would rate myself a solid A in ML/DL Coding, but a B in LLM based coding, since the libraries and functions mostly keep on changing here with the ecosystem evolving rapidly. so i would need some help in figuring out these semantics from the documentation. I'm good at the architecture and logic for the code but for this rapidly evolving codebase i'd need the documentation since the libraries are updated every now and then.

2. Key architectural components of an LLM-based chatbot

A high-level architecture for the chatbot in this project can be thought of as:

User -> UI ->LangGraph workflow -> LLM + Conversation State -> Response

(I've taken reference from the chatbot i made sometime ago, here's the link <https://github.com/KhushieSharma/multiagentlangchain> but it's the basic one)

The key components are:

1. User Interface

- The user interacts with the chatbot through the application interface.
- The application should provide a conversational interface and support streaming responses.

2. LLM

- The LLM is the core component that understands the user's query and generates a response.
- The prompt sent to the LLM contains the current user message along with relevant conversation history.

3. LangChain

- LangChain provides the building blocks for working with the LLM, prompts, messages, and model interactions.
- It helps simplify the process of connecting the application to the underlying LLM.

4. LangGraph

- LangGraph is used to define the chatbot's workflow and maintain its state.
- In a simple chatbot, the graph can have an LLM node that receives the conversation state, calls the model, and adds the generated response back to the state.

5. Conversation State / Memory

- The chatbot needs to maintain previous messages so that the LLM can understand the context of the conversation.
- For example, if a user first says "My name is khushi" and later asks "what is my name?", the chatbot can answer correctly because the previous interaction is available in its conversation state.

6. Streaming

- Instead of waiting for the complete LLM response, the application streams the response as it is generated.
- This improves the user experience and makes the chatbot feel more responsive.

This is based on a few agentic chatbots i created earlier, in these repositories:

<https://github.com/KhushieSharma/Factfinder>

<https://github.com/KhushieSharma/mini-agentic-ai-projects> (In this when you run [app.py](#) you'll find the chatbot working perfectly.)

3.What a vector database is

A vector database stores embeddings (numeric vectors produced by a model to represent the meaning of text, images, or other data) and retrieves the ones closest to a query vector. It finds items by semantic similarity rather than exact keyword or value match.

How it works?

1. A model converts each item into a vector. Queries are embedded with the same model.
2. Closeness is measured with cosine similarity, dot product, or Euclidean distance.
3. Exact search is $O(N)$, which is too slow at scale, so databases use approximate nearest neighbor (ANN) indexes that trade a little recall for large speedups. Common ones: HNSW (graph-based, fast, memory-heavy), IVF (cluster-based), product quantization (compression), and DiskANN (SSD-based).
4. Real applications also need metadata filtering (e.g: by client or date) and often hybrid search, combining vector search with keyword search (BM25), since semantic search alone misses exact terms like acronyms and product codes.

Common options are pgvector, Pinecone, Weaviate, Qdrant, Milvus, Chroma/FAISS (prototyping), Elasticsearch/OpenSearch.

Hypothetical problem:

An internal RAG assistant over 2 million document chunks (reports, methodology write-ups, client deliverables). Requirements: users see only documents for projects they are authorized to access, queries mix natural language with exact technical terms, daily inserts with occasional updates and deletes, p95 latency under 200 ms, and a small team with no dedicated infrastructure engineers.

Choice: Qdrant

- **Filtering:** Qdrant applies payload filters during HNSW traversal, not after. With strict access control, post-filtering would return empty or degraded results when top hits belong to unauthorized documents. This is the deciding factor.
- **Hybrid search:** Supports dense and sparse vectors in one collection, so keyword and semantic results can be fused (e.g., reciprocal rank fusion) without a separate search engine.
- **Scale:** 2M chunks at 1024 dimensions is about 8 GB of raw float32, which fits on one node. Quantization can reduce it further.
- **Operations:** Single binary or Docker image, with a managed cloud option. Handles updates and deletes without full re-indexing.

Why not the others??

- **pgvector:** Best if data is under 1M vectors and already in Postgres. Heavily filtered hybrid queries at 2M+ are harder to tune.
- **Pinecone:** Low ops, but costly and proprietary for a scale that doesn't need it.
- **Milvus:** Designed for billions of vectors, the distributed overhead isn't justified here.
- **Chroma/FAISS:** Suited to prototypes, lacking production-grade filtering and persistence.
- **Elasticsearch:** Reasonable if already deployed, but less cost-effective for vector search at this scale.

What would change the choice?? under 500K vectors on existing Postgres -> pgvector; no capacity for any infrastructure -> Pinecone or Qdrant Cloud; billions of vectors -> Milvus; existing Elasticsearch deployment -> extend it.